ENGINEERING PORTFOLIO / 임철규

인식 모델부터 앱·서버·배포까지

백엔드 · Android 온디바이스 AI | 기업·계약 개발 실무 약 3년 8개월 + 개인 제품 개발

직접 학습해 출시한 경험과 AI 에이전트를 활용해 제품을 운영하는 경험

직접 구축 게임봇: 데이터·라벨링 → Darknet YOLO → TFLite → Android 객체 탐지·조작 → 출시
AI 활용 개발 멍로그·블링크: 음향·눈동작 인식을 앱에 연결하고 실제 기기·출시 산출물로 확인

01 백엔드와 배포를 함께 설계합니다

Stockbot은 주가를 수집·가공·조회하는 개인 분석 도구입니다. 9개 Rust crate, tx/rx 생산·소비 분리, PostgreSQL 파티션·CTE·윈도 함수와 실행이력 조회를 연결했습니다. 공통 ARM64 빌드층과 GitHub Actions CI/CD 4경로를 직접 구성했습니다. **2-5쪽**

02 모델을 실제 기기에서 쓰게 만듭니다

Gamebot은 게임 화면의 객체와 상태를 인식해 반복 조작을 줄인 앱입니다. 에이전트 없이 데이터 준비·학습·변환·추론·조작·출시를 혼자 연결했습니다. 공통 Android 엔진을 게임별 앱으로 확장했고, 2020년 upstream 질문 기록도 남아 있습니다. **6쪽**

03 AI 활용을 구현·검증·운영으로 연결합니다

MungLog는 집의 폰으로 반려견 소리를 감지하고 밖에서 확인합니다. **BlinkRoll·Page·Game**은 ML 기반 눈 관측을 스크롤·페이지·시선 좌우 이동으로 바꿉니다. 에이전트를 활용한 개발에서 데이터 소유권, 중복 처리, 모델 적용 경계와 배포 검증을 다뤘습니다. **9-11쪽**

그 밖의 설계 경험 ColaZZang의 산책 기록·워치 입력을 KullChip의 센서 원본 수집과 ML0/ML1/ML2 분석으로 확장했습니다. Rust/JNI·온톨로지/XAI와 서버 모듈은 세대·검증 범위를 구분해 설명합니다. **7-8쪽**

GitHub · 공개 문제해결기록 · Actix 공식 예제 기여 · tb_row crate

밑줄 설명을 누르면 고정 커밋의 코드로 이동합니다. Stockbot은 운영 식별값을 바꾼 소스 사본, 다른 비공개 프로젝트는 실제 코드 발췌입니다. Stockbot 원본·개발 이력은 이력서 연락처로 초대받을 이메일을 알려 주시면 공유합니다.

수집·배치·API를 공통 계약으로 연결

여러 해의 가격·거래량과 상승·하락 흐름을 비교하기 위한 개인 리서치 도구입니다. Rust를 익혀 수집부터 브라우저 조회까지 구현했습니다.

외부 시세·종목 데이터 → Rust 수집 → Batch/Job/Step → PostgreSQL
→ 가격 구간·거래량 등 집계 → Actix API · utoipa → Yew/Rust WASM 웹

문제 | 처리 순서와 각 실행부의 책임을 분리해야 했습니다

Spring Batch와 유사한 **Batch/Job/Step 인터페이스**를 직접 구성했습니다. 수집·변환·저장 작업을 실행 절차와 나누고, 배치·스케줄러·API·웹을 별도 산출물로 관리했습니다. 9개 workspace 구성원에서 공유 모델·클라이언트·유틸리티를 추출하고 **Cargo feature로 DB·웹 의존성**을 구분했습니다.

동시성 | tx/rx 채널로 수집과 후처리·저장을 겹쳐 수행했습니다

생산자 → bounded 채널(tx/rx) → 여러 소비자를 Tokio 작업으로 나눴습니다. 가격 수집은 파일·종목 명령을 큐에 넣고 소비자가 파싱·DB 저장을 담당합니다. bound 계산은 가격 묶음을 받아 계산·upsert합니다. bound의 용량 1 채널은 **send().await로 큐가 비워질 때까지 생산 측을 대기**시킵니다. 수신 핸들을 복제해 작업을 분배하고 JoinHandle로 소비자 완료를 기다립니다.

설계 | API·DB·명령줄이 같은 실행 조건을 사용합니다

BatchParams의 배치 종류·국가·실행 상태를 JSON 직렬화와 utoipa 스키마에 함께 사용하고, to_args로 실행 인자에 연결했습니다. Actix 핸들러·공유 타입에서 OpenAPI와 Swagger UI를 구성했습니다. 열 헤더·순서·직렬화 규칙은 **tb_row trait와 derive macro**로 추출해 crate로 배포했습니다.

데이터 | 재수집·기간 조회·분석 특징을 각각 다뤘습니다

종목별 마지막 저장일부터 수집하고 **종목·날짜 기준 upsert**로 재실행 시 중복을 다뤘습니다. 날짜 RANGE 파티션과 집계 테이블을 구성했습니다. **bound**는 같은 방향이 이어지는 가격 구간이며 길이·변화율을 계산합니다. **y=mx+b**는 두 시점으로 만든 구간 특성으로, 과거 흐름을 비교하는 분석 값입니다.

다른 언어로도 검토 별도 Go 버전은 모듈별 수집과 MongoDB bulk upsert, Java/Spring 버전은 Spring Batch Tasklet·청크 Reader/Processor/Writer와 JDBC upsert를 구현했습니다. 각각의 실험 저장소이며 일괄 운영 전환으로 묶지 않습니다.

구현 근거

tx/rx 생산·소비 분리 · 계산·저장 소비자 · 가격 수집 명령
Step 절차 · 공유 API 스키마 · OpenAPI·Swagger 등록 · 파티션 DDL
공개 소스 사본 2794025 / 원본 main 18c1f8e · 상세 기록 · tb_row

2022-2025 직접 개발 / 2026 Axum·React 전환은 AI 보조 후속 작업. 후속 과제는 파티션 연말 경계·구간 꼬리 재계산 보완입니다.

공통 ARM64 기반을 서비스별로 확장

배치의 CPU 간섭을 줄이기 위해 실행 역할을 나누고, 공통 교차 빌드 환경과 산출물별 배포 경로를 구성했습니다.

공통 Rust ARM64 빌드층(툴체인·linker·sccache)
→ API / batch / scheduler 선행 빌드층 3개 → 서비스 재빌드 → 실행 이미지
웹은 공통 기반에 Trunk·WASM 도구를 더해 정적 dist 생성

문제 | 반복 빌드 환경과 운영 실행 환경이 달랐습니다

AArch64 타깃·크로스 컴파일러·네이티브 의존성을 공통 이미지에 구성했습니다. API·배치·스케줄러는 각 소스와 의존성을 선행 빌드하는 층으로 확장하고, 마지막에는 컴파일러와 분리된 실행 이미지를 만들었습니다. ARM64 libc·OpenSSL 호환 문제와 DB 복원 메모리 문제를 운영 기록에 남겼습니다.

GitHub Actions CI/CD | 산출물별 네 빌드·배포 경로

API·스케줄러: QEMU/Buildx ARM64 → ECR → 배포 묶음 S3 → CodeDeploy.
배치: ARM64 이미지 → ECR. **웹:** Docker로 WASM 빌드 → dist 추출 → S3 정적 배포.
경로 필터와 수동 실행을 두고, 백엔드에는 latest·commit 태그를 발행했습니다. 네 워크플로 모두 과거 성공 실행을 확인했습니다.

운영 | 배치를 상시 API와 분리하고 시작·종료를 제어했습니다

같은 서버의 배치 CPU 간섭 기록을 계기로 **DB / API+스케줄러 / 배치 전용**의 EC2 3역할로 나눴습니다. 스케줄러 요청 → API의 계획 ID·조건 기록 → 전용 EC2 시작 → 부팅 스크립트의 배치 실행 → 완료 API → 인스턴스 정지로 연결했습니다. S3는 웹·빌드 캐시·배포 묶음 3역할, CodeDeploy는 1앱·2그룹으로 구성한 기록이 있습니다.

구현 근거

공통 ARM64 기반 · 서비스별 빌드층 · 실행 이미지 분리 · API 배포 workflow
작업 조건 user-data · 전용 EC2 시작 · 완료 후 EC2 정지
시기: API·Actions 기반 2023년 / 서비스별 선행 빌드층 2025.11 / AI 보조 현대화 2026년

구성 수·성공 실행은 원본의 과거 기록 기준입니다. 사본은 Actions 실행을 껐습니다. API no-cache, 나머지 캐시 사용입니다. 자동 시험 게이트·무중단 배포 성과를 뜻하지 않습니다.

원천 시세를 SQL 조회 모델로 가공

여러 해의 가격·거래량을 같은 기준으로 비교하려면 원천 시계열, 가격 구간, 기간별 집계의 키와 계산 순서를 나눠야 했습니다.

종목 CODE + hist.price → 거래일 tick · price_chg · bound
→ 기간별 agg.*_data → 빈도·비중 agg.* → 조회 view → Actix API
화살표는 가공 흐름이며, 각 단계가 모두 외래키로 연결된 구조는 아닙니다.

테이블 | 데이터의 기준과 중복 경계를 키로 표현

CODE / CODE_DETAIL 종목 ID와 종목 속성. 시계열·집계에서 CODE를 참조합니다.

hist.price (code_id, dt) 종목·날짜 복합 기본키, 날짜 RANGE 파티션과 인덱스.

hist.price_chg (code_id, dt) 원천 시세의 복합키를 참조하는 거래량 변화율.

hist.bound (code_id, line, p1x) 종목·가격 종류·시작일로 방향 구간을 식별하고 날짜 분할.

agg.vol_data / vol · bound_data / bound 종목·연도·기간·통계 기준을 조합해 중간 집계와 조회 결과를 구분.

윈도 함수 | 거래일 축과 직전 거래량을 계산

저장된 시세에서 거래소별 날짜를 RANK로 순번화해 구간 길이를 거래일 축에서 다룹니다. 종목별 LAG로 직전 거래량을 가져오고 0·초기값을 분기해 변화율을 계산합니다. 누락 계산행 조회와 Rust 배치의 프로시저 CALL까지 연결했습니다.

기간 통계 | 한 해의 순위를 여러 해의 반복 빈도로 변환

월·주·분기를 UNION ALL로 묶고 최대·최소·평균 기준의 RANK를 계산합니다. CTE·윈도 함수로 여러 해에서 기준별 1위였던 기간의 반복 빈도·비중을 구하고, 공동 1위는 array_agg로 보존해 API view로 반환합니다. bound 길이·변화율의 기간 집계는 공통 PL/pgSQL 함수로 재사용했습니다.

구현 근거

시세 키·파티션 · 구간 키·FK · 집계 키
거래일 RANK · LAG 변화율 · 기간별 UNION ALL·RANK
CTE·반복 빈도·비중 · 공동 1위 array agg · 공통 집계 함수

현재 Job에서 호출되는 SQL을 기준으로 설명합니다. 별도 UNNEST·CSV/COPY 적재 실험과 구분합니다. 연말 파티션 경계·LAG 증분 첫 행·기존 bound 꼬리 계산은 보완 과제입니다.

수집 결과와 배치 실행을 웹에서 확인

외부 데이터의 형식마다 수집·파싱을 나누고, 배치가 언제 어떤 단계까지 실행됐는지 DB 이력부터 운영 화면까지 직접 연결했습니다.

Naver HTTP → JSON 계열 파싱 / KRX OTP → Excel·종목 상태 파싱
→ Job/Step 수집·저장 → system.batch / batch_step 실행이력
→ Actix GET /system/logs/batch → Yew/Rust WASM 운영 표

수집 | HTTP·파일·웹 스크래핑의 차이를 나눴습니다

Naver 시세는 종목·날짜를 지정한 HTTP 응답을 파일로 받고 날짜·OHLCV를 파싱합니다. KRX는 OTP 발급 → Excel 다운로드 → calamine Sheet1 처리 → 종목 상태 upsert로 이어집니다. 종목별 마지막 저장일을 기준으로 재수집 범위를 잡았습니다. 별도 HTML 뉴스 파서에는 날짜·매체·제목·링크 선택자를 구현했습니다.

병행 처리 | 수집과 가공·저장을 tx/rx로 분리

Tokio 작업과 async_channel::bounded를 조합했습니다. 생산 측은 파일·종목 명령이나 가격 묶음을 보내고 여러 소비자가 파싱·계산·DB 저장을 맡습니다. bound 용량 1 채널은 send().await로 생산 측을 기다리게 합니다. 가격 HTTP 요청은 종목별 순차 await가 남아 있어, 모든 요청을 동시에 실행한 구조로 설명하지 않습니다.

운영 조회 | 실행이력 DB·API·WASM을 한 흐름으로 연결

system.batch는 실행 부모, batch_step은 batch_id로 연결한 Job/Step 이력입니다. 시작·종료 시각·소요시간을 기록하고 DAO의 JOIN·정렬·count·페이지 조회를 Actix API와 WASM 표로 제공합니다. START/EXIT 실행계획과 스케줄러 heartbeat를 따로 둬 실행 조건·작업 이력·현재 시각 정보를 구분했습니다.

구현 근거

HTTP 시세 요청 · 시세 응답 파싱 · KRX OTP·Excel · 종목 상태 파싱
HTML 뉴스 선택자 · 실행이력 DDL · 이력 기록
조회 JOIN·페이지 · 조회 API · WASM 운영 표 · 실행계획 · heartbeat

별도 중앙 로그 수집 서버는 없습니다. 뉴스 모듈은 기본 main 미등록·소비자 await 누락으로 미완성입니다. 오류 시 완료 기록·자동 재시작을 모두 보장하는 운영 체계로 주장하지 않습니다.

비전 AI를 Android 앱으로 출시

2020-2021년, AI 코딩 에이전트 없이 혼자 진행했습니다. 게임 화면의 버튼·객체·상태를 인식하고 반복 조작을 줄이는 앱을 만들어 출시한 뒤 서비스를 종료했습니다.

녹화 → 프레임 추출·라벨링 → Darknet YOLO 학습 → SavedModel·TFLite
→ Android 화면 캡처 → 객체 탐지·NMS → 상태 판단 → 좌표 복원 → 접근성 조작

학습 | 기존 모델 호출을 넘어 데이터와 변환 과정을 연결했습니다

GitHub의 Darknet·변환 프로젝트를 읽고 환경을 설치했습니다. OpenCV·라벨링 도구로 프레임과 라벨을 준비하고 train/valid 목록·cfg·클래스 구성을 만들었습니다. 직접 학습한 모델을 TensorFlow SavedModel·TFLite로 변환해 앱에 넣었습니다. GPU 실행 환경과 자체 모델의 TensorFlow 추론 중 좌표·미검출 문제는 upstream에 직접 질문했습니다.

설계 | 공통 인식·조작 엔진과 게임별 앱을 나눴습니다

공통 Android 라이브러리: 화면 캡처·추론·좌표·상태 규칙·접근성 입력·종료를 담당합니다. 모델은 무압축 자산으로 패키징하고 읽기 전용 mmap으로 연결했습니다.
게임별 앱: 공통 Activity를 상속하고 Gradle 입력·출력 규격, 패키지·모델 자산·버전·서명을 분리했습니다.

병목 | 오래된 화면에 잘못 행동하지 않게 처리했습니다

라벨별 신뢰도·IoU NMS로 후보를 정리하고 추론 시간 기반 반복 관측·이전 행동으로 상태를 판단했습니다. 중간 JPEG I/O 없이 한 장 버퍼·직렬 추론으로 적체를 다루고, 모델 좌표를 실제 화면으로 복원했습니다. 자기 overlay 클릭을 막고 회전·종료 시 자원을 정리했습니다.

구현 근거

공통 엔진 소스 · SK2 앱 소스 · 실행 영상
Darknet 질문 #5710 · 2020.05 · TF/TFLite 프로젝트 질문 #103 · 2020.06
소스: common 3338a849 / sk2 ae8e3d21 · Run·Classifier·Target·TouchService·build.gradle

개발·출시 기간은 2020-2021년이며 현재 서비스는 종료됐습니다. 1인·에이전트 미사용 사실은 본인 설명과 당시 소스·외부 질문 기록을 함께 제시합니다.

가족의 산책 기록에서 센서 수집으로

같은 반려견을 번갈아 산책시키며 "어디서 무엇을 했는가"를 공유하려고 시작했습니다. 휴대폰을 꺼내는 불편을 워치 입력으로, 자동 기록의 어려움을 센서 분석으로 이어갔습니다.

ColaZZang: 워치의 산책·마커 입력 ↔ 휴대폰 GPS·Room 기록
KullChip 후기: 워치 센서 RAW → 파일 spool → 전송 → 모바일 Room commit
→ 확인 응답(ACK) → 확인된 원본만 정리

출발 | 앱 전체를 한 화면이 아니라 역할별 모듈로 나눴습니다

ColaZZang 원본은 app·wear·domain·data·sync·stopwatch·style의 7개 모듈로 구성했습니다. 워치 입력과 모바일 저장·GPS, 양방향 상태 동기화를 나눴습니다. 이후 KullChip에서는 강아지에 부착한 워치 센서로 몸짓을 기록하는 방향을 실험했습니다.

정합성 | 전송 성공과 원본 저장 완료를 구분했습니다

수집 owner와 bounded queue로 묶음 저장하고 **Room commit 후 ACK**를 보냅니다. 같은 batch ID의 다른 데이터는 거부하고 기기·세션별 순번으로 중복·누락을 구분합니다. 기기 간 시각 정렬은 왕복 중간 시각의 offset으로 처리해 원본 발생 시각·정렬 시간축을 나눴습니다. 단절은 용량 제한 spool·FIFO 재전송으로 다뤘습니다.

ML 구조 | 한 모델에 모든 의미를 맡기지 않았습니다

ML0: 자세·heading 관측이 충분한 몸 기준 변환과 중력 fallback을 나누고 품질을 남깁니다.
ML1: 공통 backbone과 몸짓 6축 head로 상태를 분리합니다.
ML2: 몸짓 6축 확률·불확실성의 시간 흐름에서 전조 후보를 다룹니다.
입력 스키마·상위 모델 hash·값의 hash로 계보를 검사합니다.

구현 근거

Cola 워치 마커→GPS 저장 · 저장 후 ACK · 기기 간 시계 정렬
ML0 관측 품질·fallback · ML2 입력 계보 · 상세 기록
비공개 원본의 정확한 발췌: ColaZZang 5babf326 / KullChip 4a9bf15e

ColaZZang은 산책 기록·워치 입력 앱, KullChip은 센서 분석으로 확장한 후속 프로젝트입니다. 자동 행동·전조 판정은 최종 실측 기준 미달로 제품 적용을 중단했습니다.

판정의 근거와 서비스 책임을 추적

센서 기록이 있어도 "어떤 원본과 모델로 이 결과가 나왔는가"를 설명할 수 있어야 했습니다. 모델 실험과 서버 구조를 만들되, 세대별 구현과 최종 채택 여부를 구분했습니다.

온디바이스 학습 | 후보 학습과 제품 반영을 분리했습니다

ML1의 공통 backbone을 고정하고 작은 축별 head를 개인화하도록 구성했습니다. 학습·추론 스키마, 데이터·가중치 hash와 독립 산책 holdout을 확인하고 checkpoint 복원·후보 승격을 분리해 **개선 여부를 검증하는 경계**를 만들었습니다. 최종 자동 판정은 제품 기준 미달로 적용을 중단했습니다.

이전 Android 세대 | Rust/JNI·로컬 검색·온톨로지/XAI

이전 android에서는 Android와 Rust를 JNI로 연결하고 LanceDB 기록·벡터 검색을 구현했습니다. 온톨로지로 원본·구간·추론·모델 버전·사용자 수정을 연결하고, XAI 설명이 실제 지표·근거를 인용하는지 검사했습니다. 후기 android-next는 **Room RAW-first**로 다시 구성한 별도 세대입니다.

과거 Rust 서버: 공통·도메인 crate 27개 + 실행 서비스 5개
admin-api / grpc-unary / grpc-relay / grpc-notifier / batch-server
공통 Docker 단계에서 빌드 → SERVICE_NAME으로 선택 → 단일 바이너리 runtime

서버 | 공통 빌드와 도메인별 정합성 경계를 만들었습니다

인증·사용자·기기·산책·동기화를 crate로 나누고 PostgreSQL 테스트 DB·workspace 빌드·시험·Protobuf drift 검사를 CI에 연결했습니다. 동기화 cursor는 사용자별 advisory transaction lock과 **GREATEST upsert로 ACK 역행**을 막았습니다. 그룹 산책 쓰기는 소유자·보호자·방 관리자 권한으로 제한했습니다.

구현 근거

5개 바이너리 공통 빌드·runtime 선택 · Protobuf 사본 drift 차단
XAI 지표·근거 ID 검사 · ML2 상위 모델·입력 hash
원본: KullChip 4a9bf15e · server/docker/Dockerfile · proto-check.yml · XaiAssistantGroundingPolicy.java · ModelCheckpointPromotionOwner.kt

이 페이지의 서버·LanceDB/XAI는 과거 세대의 구현입니다. 최종 봉인본에서는 클라우드 결합을 정리했습니다. 근거 구조·모델 평가·제품 적용의 범위를 나눠 관리한 경험입니다.

집에서 감지하고, 밖에서 확인합니다

출시·운영 중인 반려견 소리 앱입니다. 집에 두는 폰·밖에서 보는 폰·웹의 역할을 나누고, 기기 내 ML부터 서버 전달·삭제·알림까지 연결했습니다. AI 에이전트를 활용해 개발했습니다.

마이크 → Rust FFT 후보 → Android YAMNet 추론 → 폰의 원본·SQLite 기록
→ 재전송 → Rust API·PostgreSQL 이벤트 + 알림 outbox → FCM/Web Push
요청한 녹음만 R2에 단기 전달 → 다른 폰·React PWA에서 확인

ML | 감지·개인화·맥락 검색을 구분했습니다

Rust 후보 뒤 TFLite YAMNet을 실행하고 폰 자체의 재생음은 별도 gate로 억제합니다. 고정 모델의 521개 점수 특징 위 개인 분류기를 학습하며 **원본 녹음 단위**로 학습·평가를 분리해 구간·증강 자료의 누출을 검사합니다. 후보·독립 비교·적용·복원을 나눴고 Rust USearch·관계 그래프는 기존 소리를 찾는 파생 캐시로 사용합니다.

정합성 | 중복·늦은 업로드·알림 유실을 경계로 잡았습니다

원본은 폰, 서버는 24시간 메타데이터 중계, R2는 요청된 단기 녹음 복사본으로 나눴습니다. API는 **이벤트 저장 +outbox enqueue를 같은 DB 트랜잭션**에서 처리합니다. 요청별 advisory lock·저장 응답 재사용으로 중복을 다루고, worker는 **FOR UPDATE SKIP LOCKED**로 발송 작업을 점유합니다. 삭제 표식은 늦은 재전송이 기록을 되살리지 못하게 합니다.

모듈·운영 | 플랫폼·권한·배포 실패의 책임을 나눴습니다

순수 Kotlin core-viewer와 HTTP data-viewer를 나눴습니다. revision 병합으로 늦은 목록이 새 평가·만료를 되돌리지 못하게 하고, 401 재시도는 같은 요청 bytes·ID를 유지합니다. GCP·Docker Compose·Cloudflare/R2에서 DEV·운영 계약, migration/API DB 역할과 실패 시 환경 복원을 분리했습니다.

구현 근거

멱등 요청의 DB 잠금 · 현재 권한·SKIP LOCKED 점유 · 이벤트와 알림 트랜잭션
원본 단위 학습 split · revision·만료 병합 · 동일 요청의 인증 재시도
비공개 원본 d3c795b1의 정확한 발췌 · 상세 기록

10.30 출시 기록: Play 게시·기존 녹음의 두 폰 맥락 경로 확인. 개인화의 학습·비교·반영 기능과 새 개인 모델의 실제 감지 성능은 각각 별도로 검증합니다.

눈의 움직임을 제품의 입력으로 바꿉니다

BlinkRoll은 스크롤, BlinkPage는 페이지 넘김, BlinkGame은 시선으로 좌우 이동합니다. ML 기반 눈 관측을 실제 행동으로 연결하는 앱 3종을 AI 에이전트를 활용해 개발·출시했습니다.

카메라·MediaPipe/TFLite 관측 → 눈동작 해석·의도 → 버전 있는 입력 계약
→ Roll·Page Android 실행 / Game Unity 브리지 → 실행 결과 확인
공통 의미는 재사용하고 카메라 경로·제품 출력은 각 플랫폼이 소유

설계 | 공통 코드를 재사용하면서 제품 차이를 남겼습니다

eye-contract는 이벤트·의도·결과 타입, **eye-engine**은 순수 해석, **eye-android**는 카메라·모델 수명주기를 맡습니다. 제품 runtime은 권한·구성·시작·역순 종료를 관리합니다. Unity는 버전 있는 snapshot·event를 읽고 같은 이벤트 ID로 결과를 확인합니다. 원본 영상이나 모델 임계값을 게임 제어 코드에 넘기지 않습니다.

병목 | 오래된 상태와 한 번만 실행할 명령을 구분했습니다

상태 snapshot은 **LatestValueDispatcher**로 최신 값만 남기고 개별 명령은 ID·poll/confirm 대기열로 처리합니다. 추론은 **InferenceWorkBudget**의 실행 1개·준비 1개로 제한합니다. 게임의 숫자 관측 포트는 복사한 값만 전달하고 오래된 관측·얼굴 누락·눈 감김·세션 변경 때 비워 이전 결과의 조작을 막습니다.

검증 | 모델 관측·명령 접수·화면 이동은 다른 결과입니다

한 이벤트 ID의 중복 실행을 막고 실행 접수와 실제 이동 확인을 나눴습니다. 숫자형 신고를 재생하는 계약 회귀, release 변형 시험·R8·ABI·산출물 노출 검사와 지정 기기 기록을 연결했습니다. 같은 출시 AAB의 hash·서명·검사 결과를 보존해 **소스 수정부터 배포 산출물까지** 추적합니다.

구현 근거

최신 상태 전달 · 추론 예산과 permit 해제 · event ID 중복·poll/confirm
게임 숫자 관측·만료·세대 경계 · 상세 기록
비공개 원본의 정확한 발췌: BlinkRoll d2e90698 / 제품 BlinkGame 6b40f284

출시한 앱 3종과 후속 공통 시선 모델 연구는 별도입니다. 최신 Game 후보는 제출 상태로 기록했습니다. 현재 신기능·모델 실험은 중단하고 기존 구현·출시 증거를 보존하고 있습니다.

AI 활용을 검증 가능한 작업으로 연결

과거에는 문서·소스·커뮤니티로 제품을 만들었습니다. 현재는 문제와 수용 기준을 정하고 AI 에이전트를 활용해 구현·시험·출시 작업을 진행합니다.

과거 | 문서·소스·커뮤니티로 학습해 제품까지 연결

게임봇은 에이전트 없이 GitHub 프로젝트·upstream 답변을 읽고 환경·데이터·모델 변환·Android 실행을 연결했습니다. Stockbot의 원래 수집·API·Actions 구축도 직접 진행했습니다. 2026년 Axum·React 전환부터는 AI 보조 후속 작업으로 구분합니다.

현재 | 문제·경계·수용 기준을 정하고 구현에 활용

멍로그·블링크에서는 AI 에이전트를 구현·리팩터링·시험 자동화에 활용했습니다. 예를 들어 멍로그의 로컬 원본·서버 중계·삭제 경계, 블링크의 상태·명령·실행 확인을 분리하고 이를 코드와 회귀 조건으로 다룹니다. 판단 기준과 변경 범위를 정하는 역할, 제안된 결과의 채택·중단 책임을 맡았습니다.

확인 | 코드·기기 동작·배포 산출물을 대조합니다

코드·시험 결과뿐 아니라 실제 APK/AAB의 서명·ABI·환경 주소, 설치된 앱과 출시 기록을 대조합니다. 멍로그의 10.30 기록은 기존 녹음·두 폰 전달과 게시를 구분합니다. 블링크는 같은 숫자 신고의 재생과 실제 화면 이동을 구분합니다. KullChip의 자동 판정은 기준에 못 미쳐 적용을 중단했습니다.

채용 요구사항에서 확인할 수 있는 근거

RDB · SQL · 데이터 처리
Stockbot 파티션·LAG·CTE·윈도 집계 / MungLog 트랜잭션·outbox·삭제 표식

동시성 · 정합성 · 권한
Stockbot Tokio·tx/rx 생산자-소비자 / MungLog DB 잠금·outbox / KullChip ACK

모듈 · 공통화 · 배포
Stockbot ARM64·GitHub Actions CI/CD / KullChip 공통 빌드 / Blink 공통 계약

비전 AI · 온디바이스 ML
Gamebot 전체 객체 탐지 파이프라인 / 음향·눈동작 앱의 기기 내 추론

AI 에이전트 활용
구현·리팩터링·시험 자동화 → 코드·기기·출시 산출물 검증

경험 범위 기업 실무는 Java/Spring·PHP·웹·BI, 개인 제품은 Rust 백엔드와 Android 온디바이스 ML이 중심입니다. 운영 사례는 개인 서비스 규모이며, Git·CI 설계와 기업에서의 협업 경험은 구분해 설명합니다.

구현 근거

근거: 각 상세 페이지의 소스·출시 기록 및 공개 AI 활용·검증 기록
공개 기여: Actix examples PR #250 · tb_row crate